Trie

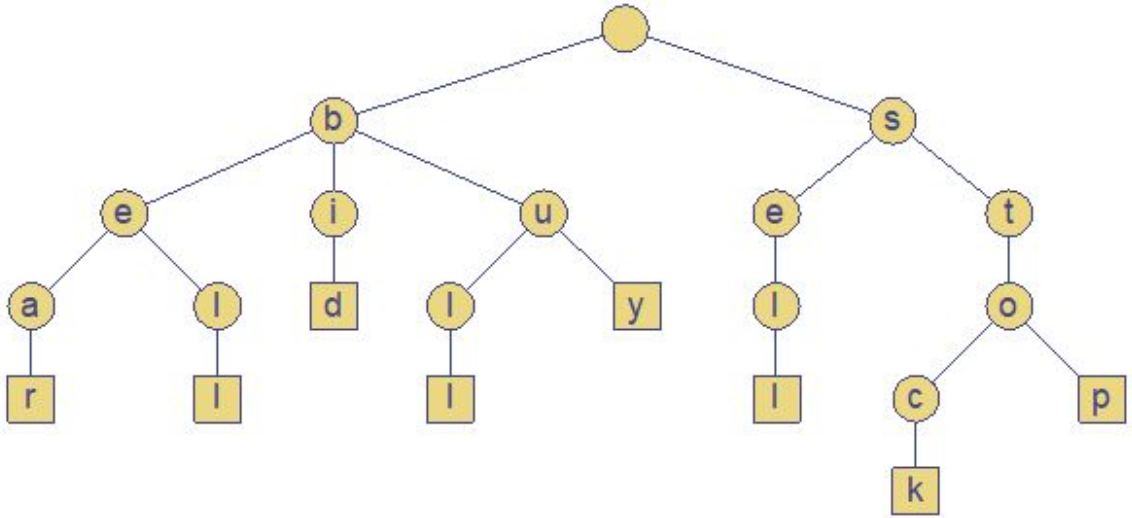
Shahriar Ivan
Lecturer, Department of CSE, IUT

Overview

- A **Trie** is a tree-based data structure used for fast information re**trie**val.
- Also known as Prefix tree or Radix tree.
- Very efficient for storing and retrieving strings
  - We want to model a set of words as a tree of some kind
  - The tree should take advantage of words containing each other to save space and time
- In this tree, the nodes are not binary. They contain potentially one outgoing edge for each possible character, so the degree is at most the alphabet size $|A|$

Trie

- The standard trie for a set of strings S is an ordered tree such that
  - Each node but the root is labeled with a character
  - The children of a node are alphabetically ordered
  - The paths from the external nodes to the root yield the strings of S
- Example: S = {bear, bell, bid, bull, buy, sell, stock, stop}



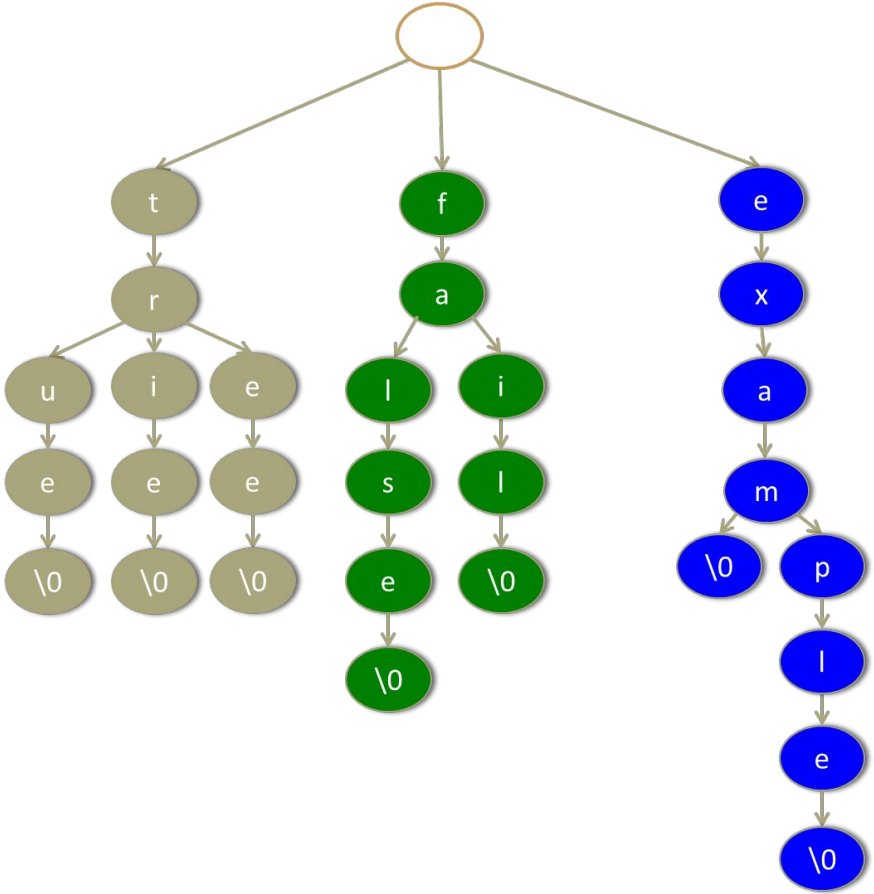
Prefix

- A prefix of a string S is a substring of S that occurs at the beginning of S
- Each node in a trie corresponds to a prefix of some strings of the set S.
  - If the same prefix occurs several times, there is only one node to represent it.
  - The root of the tree structure is the node corresponding to the empty prefix.
- What if a “Whole” string is a prefix of other string?
  - We need a way to recognize where the string ends
- There are two solutions for this:
  - We can have an explicit termination character, which is added at the end of each string, but may not occur within the string “\0” (ASCII code 0) , or
  - We can store together with each string its length.

Example

Strings:

- exam
- example
- fail
- false
- tree
- trie
- true



Find

To perform a find operation in this structure:

1. Start in the node corresponding to the empty prefix (the root).
2. Read the query string, following for each read character the outgoing pointer corresponding to that character to the next node.
3. After we read the query string, we arrived at a node corresponding to that string as prefix.
4. If the query string is contained in the set of strings stored in the trie, and that set is prefix-free, then this node belongs to that unique string.

Insert

To perform an insert operation in this structure:

1. Perform find
2. Any time we encounter a null pointer we create a new node.

The pseudocode looks as follows:

```
void insert(String s)
{
   for(every char in string s)
   {
      if(child node belonging to current char is null)
      {
         child node=new Node();
      }
      current_node=child_node;
   }
}
```

Delete

To perform a delete operation in this structure:

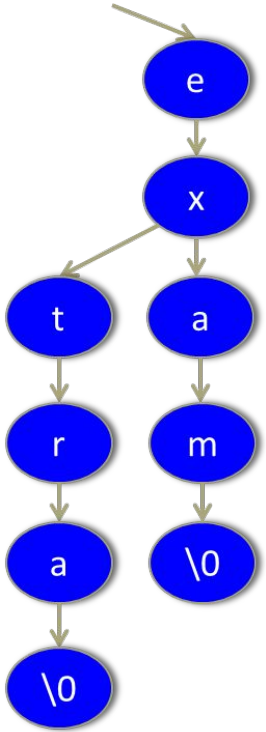
1. Perform find
2. Delete all nodes on the path from ‘\0’ to the root of the tree unless we reach a node with more than 1 child

Actually deletion in Trie is a bit more complex, as it requires checking three cases:

1. If key “k” is not present in trie, just return.
2. If key “k” is not a prefix of any other key then all the nodes starting from the root (excluding the root) to leaf node of k should be deleted.
3. If key “k” is a prefix of some other key then leaf node corresponding to k should be marked as “not a leaf node”. No node should be deleted in this case.

Deletion example

Delete “extra”



End of Lecture